Predicción de la estructura secundaria de proteínas globulares

Alberto Torrejón Aquino

25 de octubre, 2025

1 - ALGORITMO K-NN

El algoritmo *K - Nearest Neighbours (K-NN)* es un tipo de aprendizaje supervisado dentro del campo de Machine learning.

Este, también es conocido como "*Lazy learner*". Esto se debe a que no construye un modelo general durante la fase de entrenamiento, si no que se encarga de almacenar datos y, con estos datos, realiza las predicciones comparando los nuevos ejemplos con los datos existentes en el conjunto de entrenamiento.

El procedimiento básico para hacer uso de este algoritmo es el siguiente:

1. *Definir* k (número de vecinos)
2. *Calcular* distancias entre el nuevo caso y los del conjunto de entrenamiento
3. *Ordenar* esas distancias y utilizas los K más cercanos
4. *Decidir (Votar)* la clase más común entre esos vecinos
5. *Asignar* esa clase al nuevo ejemplo.

La distancia que utilizamos para este algoritmo sería la distancia euclídea. Esta distancia mide la separación entre dos vectores (x e y). La fórmula de la distancia euclidea se expresa como:

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

En este caso, el parámetro K es muy importante ya que determina cuántos vecinos se van a tener en cuenta para votar la clase final. Es adecuado elegir un K equilibrado, ya que:

K pequeño (Por ejemplo, k=1): El modelo estaría adaptandose mucho al ruido y tendería mucho al sobreajuste (overfitting), es decir, acierta en entrenamiento, pero falla en datos nuevos. No es adecuado elegir el K mínimo

K elevado (Por ejemplo, k=100): Considera demasiados vecinos (incluso de distintas clases), tendiendo a suavizar las fronteras entre clases.

Por lo tanto, para obtener un rendimiento óptimo, se prueban varios valores de k y se elige el valor con un menor error de clasificación o el mayor valor de Kappa o AUC, según el tipo de evaluación empleada.

Ventajas y desventajas del algoritmo K-NN

Fortalezas	Debilidades
Simple e intuitivo	Lento si el dataset es muy grande (esto es debido a que hay que compararlo con todos los datos de entrenamiento)
No requiere un entrenamiento complejo	Sensible al ruido y a escalas distintas (Por eso tenemos que normalizar los datos para trabajar con ellos)
Funciona bien con límites de decisión irregulares	Necesita que los datos estén bien preparados (Sin ningún valor faltante)
Puede usarse tanto para aprendizaje supervisado de regresión como de clasificación	No explica "por qué" toma una decisión, es decir, es un modelo no interpretativo.

2 - ONE-HOT ENCODING

One-hot encoding es un método que se utiliza para convertir variables categóricas en variables numéricas, de modo que puedan ser interpretadas por un algoritmo de machine learning. Consiste en crear una columna binaria por cada categoría posible, marcando con un 1 la presencia de esa categoría y con un 0 su ausencia.

En este caso, cada categoría corresponde a uno de los 20 aminoácidos posibles. Dado que la ventana de estudio abarca 17 posiciones, cada aminoácido en la secuancia se representa mediante un vector de 20 valores binarios, resultando en un total de 340 variables (17x20) para cada muestra.

De esta forma, el modelo recibe una representación numérica completa de la secuencia, que conserva la información sobre la posición y el tipo de aminoácido en cada ventana.

Para poder realizar esta codificación, R ya tiene varios paquetes prediseñados para realizar la codificación one-hot. El mas utilizado es "fastDummies", que incluye la función "dummy_cols()". Esta función, se encarga de transformar variables categóricas en nuevas columnas binarias mediante presencia (1) o ausencia(0)

```r
#Definimos la función para datasets

Codificar_onehot <- function(dataset){
  dummy_cols(dataset,
             select_columns = paste0("V",1:17),
             remove_selected_columns = TRUE)
}
```

3 - CLASIFICADOR KNN

Para poder realizar un proyecto de Machine Learning se deben realizar 5 etapas que serán cruciales seguir en orden para obtener resultados fiables. Las etapas del proyecto son las siguientes:

1. ***Recoger los datos***
2. ***Explorar y preparar los datos***
3. ***Entrenar el modelo***
4. ***Evaluar la eficiencia del modelo***
5. ***Mejorar el modelo.***

Recogida de datos (Apartado a)

Los datos proporcionados para este ejercicio se obtuvieron del Laboratorio Nacional Brookhaven, en la sección "Molecular biology (Protein Secondary Structure". Una vez descargado este dataset de este repositorio , tenemos que cargarlo en R para poder trabajar con el.

```
#Cargamos los datos en la variable data4

data4 <- read.csv("data4.csv",sep = ";", stringsAsFactors = FALSE)
```

```
#Tabla donde se muestra el número de secuencias de cada clase
table(data4$V18)
```

```
##
##    _    e    h
## 5557 1935 2508
```

Inicialmente, se observan 5557 ventanas que pertenecen a la cola, 1935 ventanas que acabarán formando una estructura secundaria Beta lamina y 2508 ventanas que formarán una estructura secundaria alpha hélice.

Explorar y preparar los datos (Apartado b)

Al visualizar el archivo, se observa el registro de aminoácidos en las columnas V1–V17 y, en la columna V18, la estructura secundaria que formaría la ventana de estudio.

El conjunto de datos no presenta valores nulos ni caracteres que puedan interferir en el análisis, por lo que no es necesaria una limpieza previa.

Sin embargo, los datos proporcionados son categóricos (letras que representan aminoácidos) y los algoritmos de machine learning requieren variables numéricas.

En el ejercicio 2, se ha desarrollado una función en R que automatiza este proceso: al introducir un dataset con las secuencias de aminoácidos, la función aplica la codificación *one-hot* utilizando el paquete fastDummies.

De esta forma, las columnas categóricas (V1–V17) se transforman en variables numéricas binarias (0 y 1), adecuadas para el posterior entrenamiento del modelo *k-NN*.

```
#Comprobamos que la función nos da como resultado "oh_enc" proporcionado en la pec

data4_onehot <- Codificar_onehot(data4)
```

El documento nuevo que genera "data4_onehot" coincide con el documento proporcionado "oh_enc" en cuanto a presencia y ausencia de aminoácidos, pero, si inspeccionamos el documento que nos genera, nos mantiene V18, que sigue siendo una variable categórica. Necesitamos extraerla y guardarla en una variable para poder tener los datos completamente procesados

```
v18 <- data4_onehot$V18 #Esta va a ser nuestra variable dependiente
data4_onehot <- subset(data4_onehot, select = -V18)
```

Entrenar el modelo multiclase (Apartado c y d)

Una vez preparado el conjunto de datos, el siguiente paso consiste en dividirlo en dos subconjuntos, uno para el entrenamiento del modelo y otro para la evaluación de su rendimiento.

Para garantizar la reproducibilidad, se fija una semilla aleatoria (4671) y se selecciona aleatoriamente un 67% de las observaciones para el conjunto de entrenamiento (training set), dejando el 33% restante para el conjunto de prueba (test set)

```
set.seed(4671)

# Variable objetivo
y <- factor(make.names(v18)) #Tiene caracteres complejos

# Variable predictora
x <- data4_onehot

# Dividimos el conjunto en entrenamiento (67%) y test (33%)
indices <- sample(seq_len(nrow(x)), size = floor(0.67 * nrow(x)))

X_train <- x[indices, , drop = FALSE]
X_test  <- x[-indices, , drop = FALSE]
y_train <- y[indices]
y_test  <- y[-indices]
```

Una vez fijada la semilla y diferenciados los conjuntos de entrenamiento y prueba, es necesario transformar los datos a formato numérico matricial para que la función knn() del paquete class pueda utilizarlos correctamente.

El algoritmo k-NN calcula distancias entre observaciones y, por tanto, solo acepta variables numéricas organizadas en forma de matriz. Por este motivo, se emplean las funciones data.matrix() y as.matrix(), que garantizan que los datos tengan el formato adecuado para el cálculo de distancias euclídeas.

```
# Convertimos a matrices numéricas
X_train <- as.matrix(data.matrix(X_train))
X_test  <- as.matrix(data.matrix(X_test))

# Construimos el modelo para los distintos valores de K
Knei_1 <- knn(train = X_train, test = X_test, cl = y_train, k = 1)
Knei_3 <- knn(train = X_train, test = X_test, cl = y_train, k = 3)
Knei_5 <- knn(train = X_train, test = X_test, cl = y_train, k = 5)
Knei_7 <- knn(train = X_train, test = X_test, cl = y_train, k = 7)
Knei_11 <- knn(train = X_train, test = X_test, cl = y_train, k = 11)
```

Evaluar modelo multiclase

Para evaluar el rendimiento del modelo k-NN en la clasificación multiclase, se utiliza la matriz de confusión, que permite comparar las clases reales con las predichas por el modelo.

Este análisis muestra de forma detallada los aciertos y errores de clasificación en cada categoría (Coil, Beta lámina y Alfa hélice), permitiendo identificar en qué tipos de estructura el modelo presenta mayor o menor precisión.

A partir de esta matriz, la función confusionMatrix() del paquete caret calcula métricas globales como el Accuracy (proporción total de aciertos) y el índice Kappa, que mide el grado de concordancia entre las predicciones y los valores reales más allá del azar.

Estos indicadores permiten comparar el rendimiento del modelo para diferentes valores de k y seleccionar el parámetro que proporcione el mejor equilibrio entre precisión y estabilidad.

```r
#K=1

confusionMatrix(Knei_1, y_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    e    h   X_
##         e   410   69  153
##         h    54  577  162
##         X_  158  172 1545
##
## Overall Statistics
##
##                Accuracy : 0.7673
##                  95% CI : (0.7525, 0.7816)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.6016
##
##  Mcnemar's Test P-Value : 0.5302
##
## Statistics by Class:
##
##                      Class: e Class: h Class: X_
## Sensitivity            0.6592   0.7054    0.8306
## Specificity            0.9171   0.9130    0.7708
## Pos Pred Value         0.6487   0.7276    0.8240
## Neg Pred Value         0.9205   0.9039    0.7789
## Prevalence             0.1885   0.2479    0.5636
## Detection Rate         0.1242   0.1748    0.4682
## Detection Prevalence   0.1915   0.2403    0.5682
## Balanced Accuracy      0.7881   0.8092    0.8007
```

```r
#K=3

confusionMatrix(Knei_3, y_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    e    h   X_
##         e   274   81  157
##         h    58  418  191
##         X_  290  319 1512
##
## Overall Statistics
##
##                Accuracy : 0.6679
##                  95% CI : (0.6515, 0.6839)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : < 2.2e-16
```

```
##
##                   Kappa : 0.4052
##
##  Mcnemar's Test P-Value : 2.825e-16
##
## Statistics by Class:
##
##                      Class: e Class: h Class: X_
## Sensitivity           0.44051   0.5110    0.8129
## Specificity           0.91113   0.8997    0.5771
## Pos Pred Value        0.53516   0.6267    0.7129
## Neg Pred Value        0.87518   0.8481    0.7048
## Prevalence            0.18848   0.2479    0.5636
## Detection Rate        0.08303   0.1267    0.4582
## Detection Prevalence  0.15515   0.2021    0.6427
## Balanced Accuracy     0.67582   0.7053    0.6950
```

```
#K=5

confusionMatrix(Knei_5, y_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    e    h   X_
##         e   214   80  130
##         h    66  322  190
##         X_  342  416 1540
##
## Overall Statistics
##
##                Accuracy : 0.6291
##                  95% CI : (0.6123, 0.6456)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : 1.289e-14
##
##                   Kappa : 0.313
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: e Class: h Class: X_
## Sensitivity           0.34405  0.39364    0.8280
## Specificity           0.92158  0.89686    0.4736
## Pos Pred Value        0.50472  0.55709    0.6701
## Neg Pred Value        0.85814  0.81778    0.6806
## Prevalence            0.18848  0.24788    0.5636
## Detection Rate        0.06485  0.09758    0.4667
## Detection Prevalence  0.12848  0.17515    0.6964
## Balanced Accuracy     0.63282  0.64525    0.6508
```

```r
#K=7

confusionMatrix(Knei_7, y_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    e    h   X_
##         e   185   69  112
##         h    62  289  185
##         X_  375  460 1563
##
## Overall Statistics
##
##                Accuracy : 0.6173
##                  95% CI : (0.6004, 0.6339)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : 2.339e-10
##
##                   Kappa : 0.2769
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: e Class: h Class: X_
## Sensitivity           0.29743  0.35330    0.8403
## Specificity           0.93241  0.90048    0.4201
## Pos Pred Value        0.50546  0.53918    0.6518
## Neg Pred Value        0.85106  0.80861    0.6707
## Prevalence            0.18848  0.24788    0.5636
## Detection Rate        0.05606  0.08758    0.4736
## Detection Prevalence  0.11091  0.16242    0.7267
## Balanced Accuracy     0.61492  0.62689    0.6302
```

```r
#K=11

confusionMatrix(Knei_11, y_test)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    e    h   X_
##         e   151   46   72
##         h    53  214  142
##         X_  418  558 1646
##
## Overall Statistics
##
##                Accuracy : 0.6094
##                  95% CI : (0.5925, 0.6261)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : 5.584e-08
```

```
##
##                   Kappa : 0.2282
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: e Class: h Class: X_
## Sensitivity           0.24277  0.26161    0.8849
## Specificity           0.95594  0.92143    0.3222
## Pos Pred Value        0.56134  0.52323    0.6278
## Neg Pred Value        0.84461  0.79108    0.6844
## Prevalence            0.18848  0.24788    0.5636
## Detection Rate        0.04576  0.06485    0.4988
## Detection Prevalence  0.08152  0.12394    0.7945
## Balanced Accuracy     0.59935  0.59152    0.6036
```

Mejorar el modelo multiclase

En esta etapa, se analizan las posibles mejoras del modelo k-NN multiclase con el objetivo de optimizar su rendimiento.

Una primera estrategia consiste en ajustar el valor del parámetro k, comparando los resultados obtenidos (Accuracy y Kappa) para los distintos valores evaluados. Generalmente, valores intermedios de k (como 3 o 5 en este caso) proporcionan un mejor equilibrio entre precisión y estabilidad, evitando tanto el sobreajuste asociado a k = 1 como la excesiva generalización de k elevados.

También, se podría aplicar validación cruzada o partición estratificada para garantizar que la distribución de las clases se mantenga en los conjuntos de entrenamiento y prueba, reduciendo el impacto del azar sobre las métricas del modelo y obteniendo una estimación más fiable de su rendimiento general.

Comentario y discusión del modelo multiclase con parámetros (apartado f)

En la clasificación multiclase, el modelo k-NN logra distinguir con eficacia las tres estructuras secundarias, aunque el rendimiento depende notablemente del valor de k.

El mejor resultado se obtiene con k = 1, con un Accuracy cercano al 0.77 y un Kappa de 0.60, lo que indica una buena concordancia entre predicción y realidad.

A medida que k aumenta, el modelo pierde precisión: el error sube y el índice Kappa disminuye, reflejando una pérdida de capacidad discriminativa por el efecto de suavizado.

Las clases más confundidas son *"e"* (lámina beta) y *"h"* (hélice alfa), mientras que la clase *"_"* (Coil) se clasifica correctamente en la mayoría de los casos (tiene sentido al ser la más probable).

Por tanto, k = 1 ofrece el rendimiento más alto, mientras que k = 3–5 logran un compromiso aceptable entre exactitud y estabilidad.

Entrenar el modelo binario (Apartado e)

Nuestra variable objetivo puede reinterpretarse en dos categorías principales: "Coil" (aminoácidos de cola) y "Non-Coil", que agrupa las estructuras alfa hélice y beta lámina. Esta reclasificación permite simplificar

el problema a un modelo de clasificación binaria, facilitando así la evaluación del desempeño mediante una matriz de confusión. A través de esta matriz, es posible analizar la calidad del modelo considerando tanto las predicciones correctas e incorrectas, como la relación entre los casos positivos y negativos reales y los predichos por el modelo. Modificamos primero la variable predictora

```
v18_binario <- v18

v18_binario[v18_binario %in% c("e", "h")] <- "NC"
v18_binario[v18_binario == "_"] <- "C"

v18_binario <- as.factor(v18_binario)


table(v18_binario)
```

```
## v18_binario
##    C   NC
## 5557 4443
```

Observamos que, esta agrupación es correcta ya que NC pasa a ser la suma de e y h de la variable V18 anterior (apartado a)

Una vez modificada la variable repetimos el proceso anterior para entrenar nuestro modelo con esta nueva variable

```
set.seed(4671)

# Variable objetivo
y <- v18_binario

# Variable predictora
x <- data4_onehot

# Dividimos el conjunto en entrenamiento (67%) y test (33%)
indices <- sample(seq_len(nrow(x)), size = floor(0.67 * nrow(x)))

X_train <- x[indices, , drop = FALSE]
X_test  <- x[-indices, , drop = FALSE]
y_train <- y[indices]
y_test  <- y[-indices]

# Convertimos a matrices numéricas
X_train <- as.matrix(data.matrix(X_train))
X_test  <- as.matrix(data.matrix(X_test))

# Construimos el modelo para los distintos valores de K
Knei_binario_1<-knn(train = X_train, test = X_test, cl = y_train, k = 1)
Knei_binario_3<-knn(train = X_train, test = X_test, cl = y_train, k = 3)
Knei_binario_5<-knn(train = X_train, test = X_test, cl = y_train, k = 5)
Knei_binario_7<-knn(train = X_train, test = X_test, cl = y_train, k = 7)
Knei_binario_11<-knn(train = X_train, test = X_test, cl = y_train, k = 11)
```

Evaluar modelo binario (Apartado e)

Una vez entrenado el modelo KNN binario, se procede a evaluar su rendimiento mediante diferentes métricas específicas para problemas de clasificación binaria.

En primer lugar, se utiliza la matriz de confusión, que permite analizar de forma detallada el número de aciertos y errores del modelo al distinguir entre las dos clases: Coil (C) y Non-Coil (NC). A partir de esta matriz se obtienen indicadores globales como la precisión (Accuracy) y el índice Kappa, que miden la concordancia entre las predicciones y los valores reales.

```
#K=1

confusionMatrix(Knei_binario_1, y_test,positive = "NC")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    C   NC
##         C  1530  320
##         NC  330 1120
##
##                Accuracy : 0.803
##                  95% CI : (0.789, 0.8165)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.5999
##
##  Mcnemar's Test P-Value : 0.7241
##
##             Sensitivity : 0.7778
##             Specificity : 0.8226
##          Pos Pred Value : 0.7724
##          Neg Pred Value : 0.8270
##              Prevalence : 0.4364
##          Detection Rate : 0.3394
##    Detection Prevalence : 0.4394
##       Balanced Accuracy : 0.8002
##
##        'Positive' Class : NC
##
```

```
#K=3

confusionMatrix(Knei_binario_3, y_test,positive = "NC")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    C   NC
##         C  1427  507
##         NC  433  933
##
```

```
##                Accuracy : 0.7152
##                  95% CI : (0.6994, 0.7305)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.4175
##
##  Mcnemar's Test P-Value : 0.01727
##
##             Sensitivity : 0.6479
##             Specificity : 0.7672
##          Pos Pred Value : 0.6830
##          Neg Pred Value : 0.7378
##              Prevalence : 0.4364
##          Detection Rate : 0.2827
##    Detection Prevalence : 0.4139
##       Balanced Accuracy : 0.7076
##
##        'Positive' Class : NC
##
```

```r
#K=5

confusionMatrix(Knei_binario_5, y_test,positive = "NC")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    C   NC
##         C  1402  588
##         NC  458  852
##
##                Accuracy : 0.683
##                  95% CI : (0.6668, 0.6989)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.349
##
##  Mcnemar's Test P-Value : 6.646e-05
##
##             Sensitivity : 0.5917
##             Specificity : 0.7538
##          Pos Pred Value : 0.6504
##          Neg Pred Value : 0.7045
##              Prevalence : 0.4364
##          Detection Rate : 0.2582
##    Detection Prevalence : 0.3970
##       Balanced Accuracy : 0.6727
##
##        'Positive' Class : NC
##
```

```r
#K=7

confusionMatrix(Knei_binario_7, y_test,positive = "NC")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    C   NC
##         C  1393  627
##         NC  467  813
##
##                Accuracy : 0.6685
##                  95% CI : (0.6521, 0.6845)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3175
##
##  Mcnemar's Test P-Value : 1.531e-06
##
##             Sensitivity : 0.5646
##             Specificity : 0.7489
##          Pos Pred Value : 0.6352
##          Neg Pred Value : 0.6896
##              Prevalence : 0.4364
##          Detection Rate : 0.2464
##    Detection Prevalence : 0.3879
##       Balanced Accuracy : 0.6568
##
##        'Positive' Class : NC
##
```

```r
#K=11

confusionMatrix(Knei_binario_11, y_test,positive = "NC")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    C   NC
##         C  1417  670
##         NC  443  770
##
##                Accuracy : 0.6627
##                  95% CI : (0.6463, 0.6789)
##     No Information Rate : 0.5636
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3019
##
##  Mcnemar's Test P-Value : 1.251e-11
##
##             Sensitivity : 0.5347
```

```
##             Specificity : 0.7618
##          Pos Pred Value : 0.6348
##          Neg Pred Value : 0.6790
##              Prevalence : 0.4364
##          Detection Rate : 0.2333
##    Detection Prevalence : 0.3676
##       Balanced Accuracy : 0.6483
##
##        'Positive' Class : NC
##
```

Además, se incorporan métricas complementarias como la curva ROC (Receiver Operating Characteristic), que representa la relación entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR), y el área bajo la curva (AUC), que resume la capacidad del modelo para discriminar entre ambas clases. Cuanto mayor sea el valor del AUC (cercano a 1), mejor será el desempeño del modelo al separar las categorías "C" y "NC".

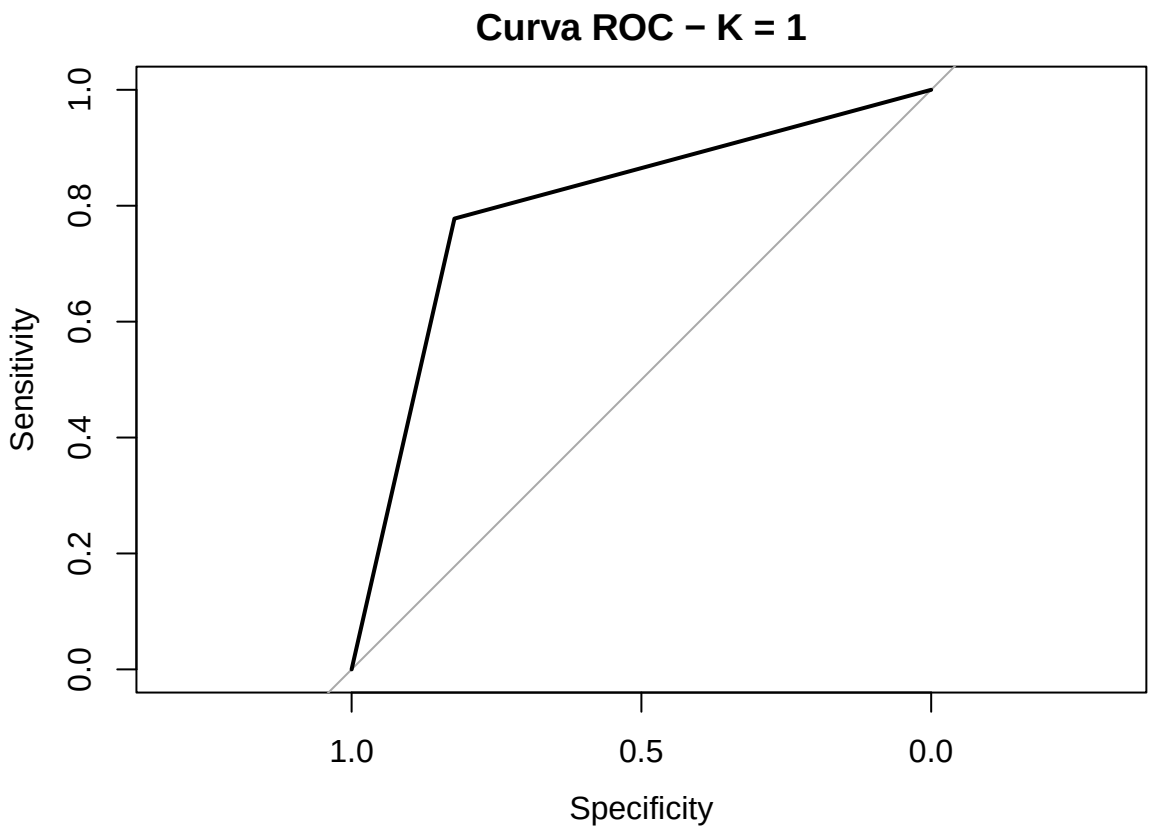
```r
# Convertimos clases a numéricas (1 = NC, 0 = C)
y_test_num <- ifelse(y_test == "NC", 1, 0)

# Función para calcular y graficar ROC/AUC
roc_auc <- function(pred, y_test_num, k) {
  pred_num <- ifelse(pred == "NC", 1, 0)
  roc_obj <- roc(y_test_num, pred_num)
  plot(roc_obj, main = paste("Curva ROC - K =", k))
  print(paste("AUC para K =", k, ":", round(auc(roc_obj), 3)))
}

# Calcular para cada modelo
roc_auc(Knei_binario_1, y_test_num, 1)
```

```
## Setting levels: control = 0, case = 1
```

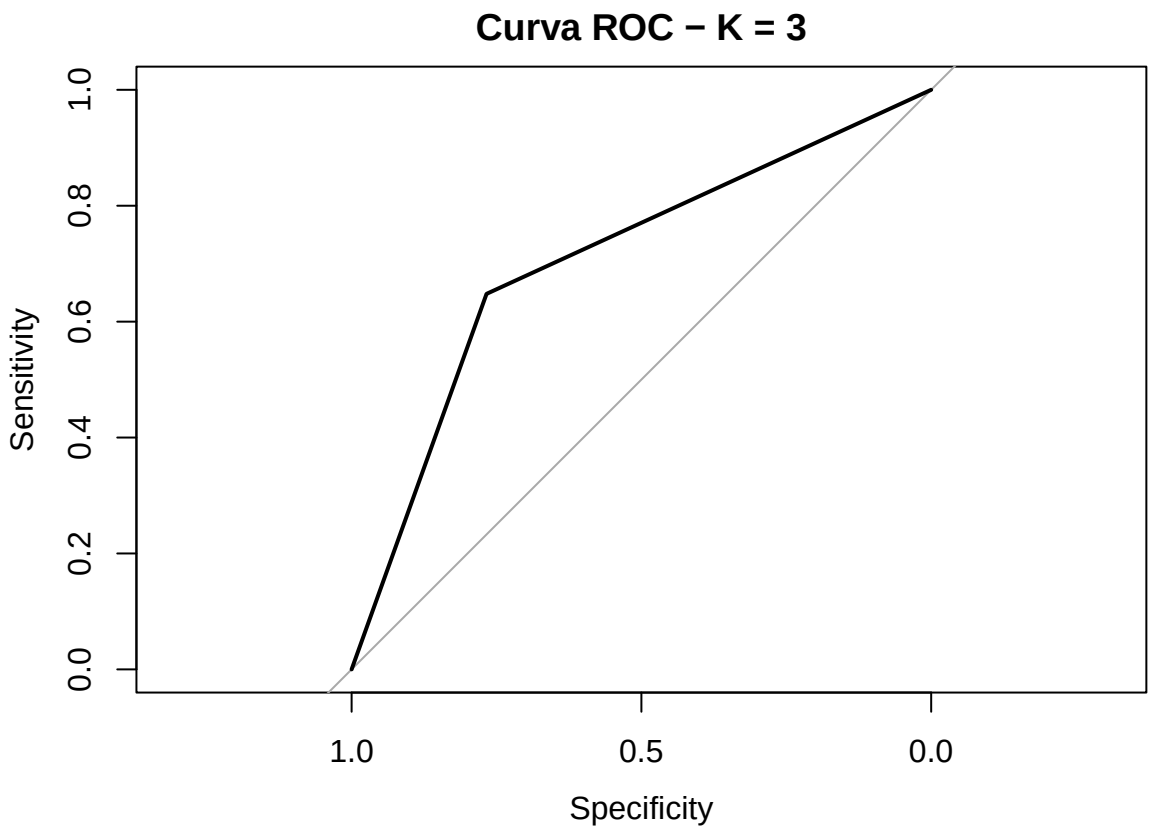
```
## Setting direction: controls < cases
```




```
## [1] "AUC para K = 1 : 0.8"
```

```
roc_auc(Knei_binario_3, y_test_num, 3)
```

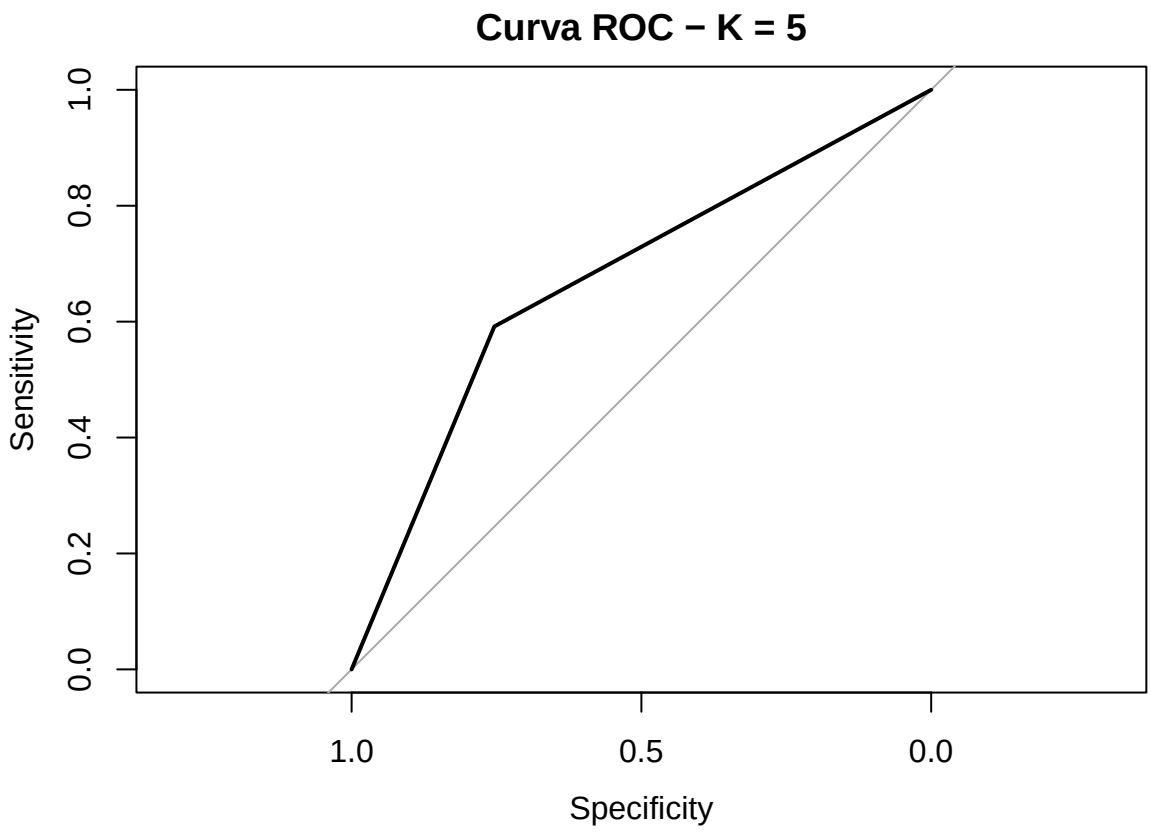
```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```




```
## [1] "AUC para K = 3 : 0.708"
```

```
roc_auc(Knei_binario_5, y_test_num, 5)
```

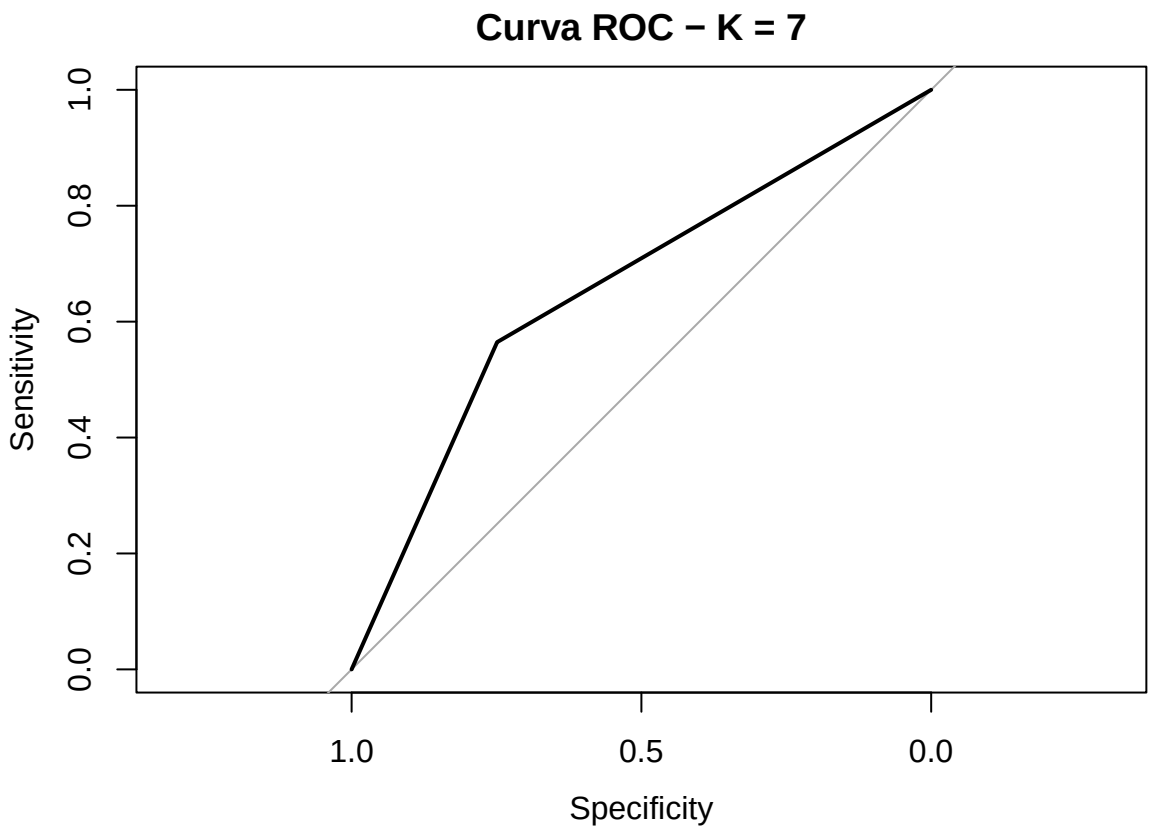
```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```




```
## [1] "AUC para K = 5 : 0.673"
```

```
roc_auc(Knei_binario_7, y_test_num, 7)
```

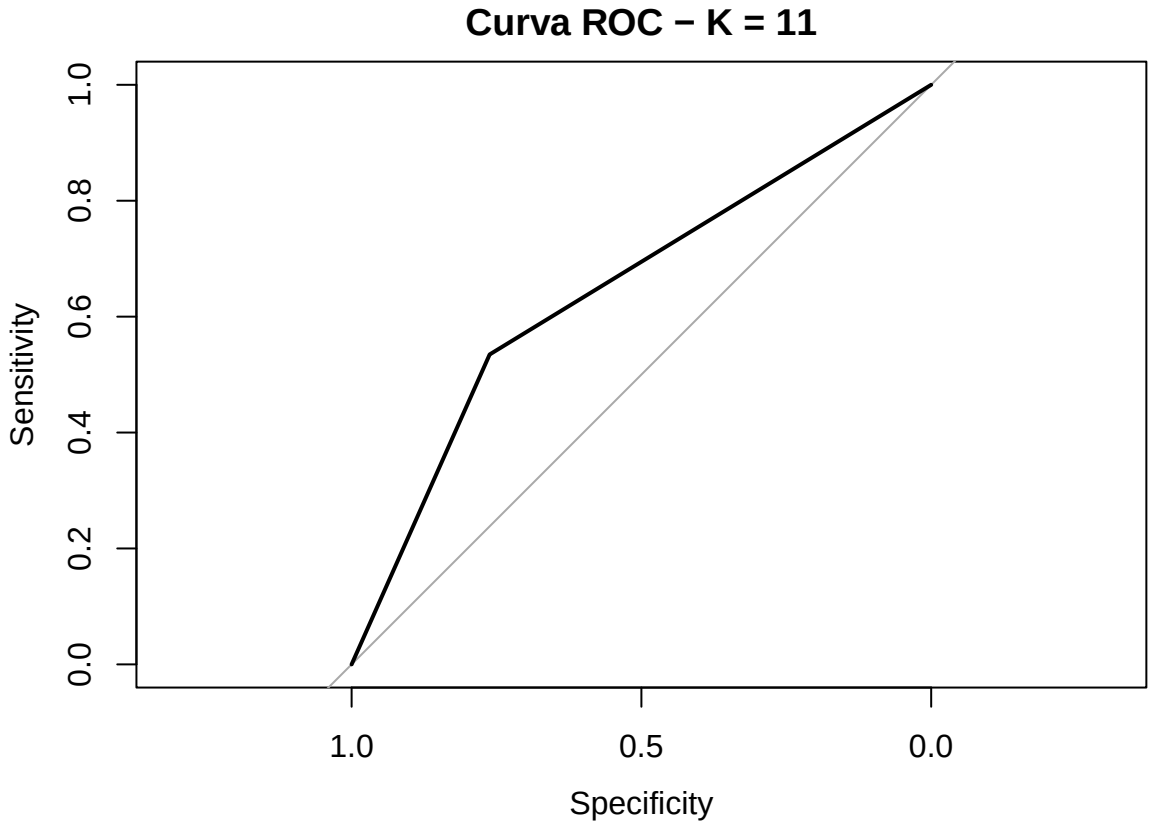
```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```




```
## [1] "AUC para K = 7 : 0.657"
```

```
roc_auc(Knei_binario_11, y_test_num, 11)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```




```
## [1] "AUC para K = 11 : 0.648"
```

Mejorar el modelo (Apartado e)

En esta fase se plantean posibles mejoras para el modelo k-NN binario, con el fin de optimizar su capacidad de clasificación entre las estructuras "Coil" (C) y "Non-Coil" (NC).

A partir de los resultados obtenidos, se observa que el modelo con k = 1 presenta el valor de AUC más alto (0.8), lo que indica un buen desempeño inicial. No obstante, este valor de k tan bajo puede implicar un cierto sobreajuste al conjunto de entrenamiento, por lo que sería recomendable evaluar otras configuraciones que mantengan un equilibrio adecuado entre sensibilidad y especificidad.

Una primera estrategia de mejora sería ajustar el parámetro k utilizando técnicas de validación cruzada, lo que permitiría identificar el valor óptimo que maximice el AUC promedio y mejore la capacidad de generalización del modelo.

Además, sería conveniente normalizar o estandarizar las variables predictoras, dado que el algoritmo k-NN se basa en distancias y puede verse afectado si las escalas de las variables difieren entre sí.

Por otro lado, podría aplicarse una partición estratificada para asegurar que ambas clases ("C" y "NC") estén representadas de manera proporcional en los conjuntos de entrenamiento y prueba, evitando así posibles sesgos en la evaluación del rendimiento.

Comentario y discusión del modelo binario con parámetros (apartado f)

En la clasificación binaria, el rendimiento mejora en general: con k = 1 se obtiene el AUC más alto (0.8), junto con un Accuracy 0.80 y un Kappa 0.60, lo que indica una buena capacidad de discriminación entre las clases Coil (C) y Non-Coil (NC).
El AUC (Área Bajo la Curva ROC) mide la habilidad del modelo para distinguir correctamente entre ambas clases; en este caso, un valor de 0.8 refleja que el modelo tiene una probabilidad del 80% de asignar una puntuación más alta a un ejemplo de la clase NC que a uno de la clase C.
El número de falsos positivos (330) y falsos negativos (320) se mantiene equilibrado, lo que sugiere un modelo bien calibrado en cuanto a sensibilidad y especificidad.

Al incrementar el valor de k, el modelo pierde poder discriminativo: el AUC desciende progresivamente (0.708 para k = 3, 0.673 para k = 5, 0.657 para k = 7, y 0.648 para k = 11).
Este descenso del AUC indica que el modelo se vuelve menos capaz de separar ambas clases, mientras que los falsos positivos y falsos negativos aumentan, elevando el error global.
En consecuencia, valores altos de k producen un modelo más general pero menos preciso, lo que compromete su capacidad de detección fina de las estructuras Non-Coil.

```
# CITAS BIBLIOGRÁFICAS
```

@book{lantz2023machine, title={Machine learning with R: learn techniques for building and improving machine learning models, from data preparation to model tuning, evaluation, and working with big data}, author={Lantz, Brett}, year={2023}, publisher={Packt Publishing Ltd} }

Sejnowski, T. & Qian, N. (1988). Molecular Biology (Protein Secondary Structure) [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5SP4F. “‘